Step-level annotation guidelines

Guideline version: 1.0 (draft for pilot)

Unit: one non-empty line of a model response, including any final-answer line. Read the complete question and response before labeling each line. The line boundary is a tooling convention; do not split a calculation expression that appears on one line.

Goal

Identify which response line first makes the reasoning materially invalid, unsupported, irrelevant, or misreported. Judge the reasoning, not whether the response agrees with the source's model-correctness flag. A response can reach the right answer through invalid reasoning; annotate the first invalid step anyway.

For each line, choose one status:

- **correct** — mathematically and logically valid, supported by the question or earlier valid steps, and relevant to solving the question. A harmless restatement can remain correct.
- **first_error** — the earliest line that is materially wrong, unsupported, irrelevant, or misreports the answer. Assign exactly one error type to this line.
- **downstream_error** — an incorrect line after the first error that carries forward or depends on the earlier error. Do not assign another error type; the dataset focuses its taxonomy on the first failure.
- **uncertain** — the line cannot be judged reliably from the prompt and trace. Use sparingly and explain the uncertainty in the rationale.

If a later line introduces a separate problem after the first error, still use `downstream_error`: only the earliest failure receives a type. Earlier uncertain lines do not prevent a later, clearly invalid line from being marked `first_error`. If no line is incorrect, there is no first-error label.

Error taxonomy (apply only to `first_error`)

1. **`arithmetic_error`** — a numerical operation is computed incorrectly (for example, `15 - 4 = 10`). If the arithmetic is correct but its units or scale are mishandled, use `unit_or_scale_error`.
2. **`misread_or_omission`** — the line misinterprets wording or leaves out a relevant quantity, condition, or requested target (for example, ignoring eggs used for baking).
3. **`invalid_inference`** — the stated premises are available, but the conclusion does not follow from them (for example, adding two independent rates rather than applying them over time).
4. **`unsupported_claim`** — the line invents a fact or assumption not stated in or derivable from the problem (for example, adding an unstated tax).
5. **`unit_or_scale_error`** — a conversion, unit, percentage, rate, or scale factor is applied incorrectly even if the arithmetic operation itself is carried out correctly.
6. **`irrelevant_or_redundant`** — the line is a non sequitur or introduces an irrelevant detour that materially makes the solution unsound. Do not mark harmless explanation, a valid check, or simple repetition as an error.
7. **`answer_reporting_error`** — all preceding reasoning is valid, but the final answer is missing, uses the wrong requested quantity, or contradicts the valid derivation. If a wrong final value merely carries forward an earlier mistake, mark it `downstream_error` instead.

Tie-breakers

- Pure numerical miscalculation → `arithmetic_error`; a dimensional, conversion, or rate interpretation mistake → `unit_or_scale_error`.
- Misunderstood or omitted prompt text → `misread_or_omission`; a conclusion that fails despite correctly understood premises → `invalid_inference`.
- An invented premise → `unsupported_claim`; a bad transformation of stated premises → `invalid_inference`.
- If a line contains multiple issues, choose the category closest to the earliest cause of invalidity.

- Use the reference answer as a check, not as a substitute for evaluating the line. If the reference appears inconsistent with the question, flag it in the rationale and do not force agreement with it.

Worked example (synthetic)

Question: A box has 12 pencils. Alice adds 3 pencils, then gives away 4. How many pencils remain?

1. `12 + 3 = 15` — **correct**.
2. `15 - 4 = 10` — **first_error**, `arithmetic_error` (the result should be 11).
3. `Answer: 10` — **downstream_error** (it carries forward the earlier result; do not assign a second type).

If line 2 instead read `15 - 4 = 11` and line 3 said `Answer: 10`, line 3 would be the `first_error` with `answer_reporting_error`.

Annotation procedure and quality controls

1. Read the prompt, reference answer, prior steps, current step, and following steps. Do not use or infer the model identity or upstream `answer_correct` flag; they are intentionally hidden in the annotation interface.
2. Label every step. Treat `<<calculation>>` spans as model text, not independently verified labels.
3. Mark the earliest materially invalid line `first_error`, and only that line gets a taxonomy label. Mark downstream incorrect lines as `downstream_error`.
4. Add a short rationale for `uncertain` cases and for taxonomy choices that another annotator could reasonably dispute.
5. Annotators work independently on the double-annotated subset. Do not discuss examples until both initial passes are complete.
6. Preserve each annotator's raw file. Adjudication, if performed, is a separately versioned layer and never replaces source judgments.
7. After a pilot, record specific disagreements, revise this document with a new version number, and run a fresh blind pass before comparing the new agreement score. Do not claim improvement unless the same sampling and agreement procedure is used and the measured results support it.

Cohen's kappa on the four UI statuses (`correct`, `first_error`, `downstream_error`, `uncertain`) is the primary step-status agreement statistic. Always publish raw agreement, label prevalence, matched-item count, guideline version, and kappa together: kappa can be unstable or low under skewed label prevalence or small samples. Error-type agreement is conditional on both annotators identifying the same step as the first error and must include that denominator.